NLP Practical Assignment

Shivam Vishwakarma
Natural Language Processing
Roll no: 6709

Part-of-Speech Tagging and Named Entity Recognition (25 Marks)

1. Collect any news article (minimum 300 words).
2. Perform:
    - Sentence tokenization
    - POS tagging
    - Named Entity Recognition
3. Identify:
    - Most common noun phrases
    - All named entities (Person, Location, Organization)
4. Visualize a parse tree for at least 5 sentences.

Use either **NLTK** or **spaCy**.

Code

```
import spacy
import nltk
from nltk import Tree
from collections import Counter

# Load spaCy model
nlp = spacy.load("en_core_web_sm")

# Paste the full text of the article here as a single string
article_text = """Summary of Operation SINDOOR
On 22 April, Pakistan based terrorist group “The Resistance Front” (TRF)
perpetrated a devastating attack in Pahalgam, India. In this attack 26
innocent tourists were killed in cold blood after segregating them based
on their religion. TRF is an offshoot of the well known Pakistan backed
terrorist organisation Lashkar-e-Taiba. Post the attack, TRF claimed the
responsibility for this attack not once but twice within a few hours.
Pakistan’s subsequent refusal to acknowledge or curb these terrorist
networks compelled India to take a responsible but resolute action. In
response, on the night of 7-8 May 2025, the Government of India executed
“Operation Sindoor”. The response was non escalatory, precise and targeted
terrorist training camps at nine different locations within Pakistan and
```

```
Pakistan Occupied Kashmir. No military targets were engaged. However, in
the early hours of 8 May, Pakistan, in an escalated response launched
coordinated drone and missile strikes targeting over a dozen Indian
military installations across the Northern and Western theatres, including
Srinagar, Jammu, Pathankot, Amritsar, Ludhiana, Bathinda and Bhuj. India's
robust Integrated Counter-drone Grid and layered Air Defence systems
intercepted these attacks, recovering debris conclusively traced to
Pakistani origin.
Following these provocations, India conducted precision strikes against
Pakistani Air Defence systems at a number of locations in Pakistan. These
strikes were deliberately confined to the neutralization of systems that
had facilitated the earlier Pakistani assault and were executed under the
guiding principle of "equal intensity in the same domain." By targeting
only those installations directly involved in the aggression, India
balanced the imperative of deterrence with its overarching commitment to
de-escalation. Concurrently, along the Line of Control in Jammu & Kashmir,
Pakistan escalated to unprovoked mortar and heavy-calibre artillery fire
into civilian areas in which sixteen innocent lives were lost, including
three women and five children. Here too, India was compelled to respond in
equal proportion with mortar and artillery fire. Indian Armed forces
reiterate their commitment to non-esclation, however, any attempts by
Pakistan to escalate will be responded firmly.
"""
# 1. Sentence Tokenization
doc = nlp(article_text)
sentences = list(doc.sents)

print("------ Sentence Tokenization ------")
for i, sent in enumerate(sentences, 1):
    print(f"{i}. {sent.text}")

# 2. POS Tagging
print("\n------ POS Tagging ------")
for token in doc:
    print(f"{token.text}\t{token.pos_}")

# 3. Named Entity Recognition
print("\n------ Named Entities ------")
for ent in doc.ents:
    print(f"{ent.text}\t{ent.label_}")
```

```python
# 4. Most Common Noun Phrases
noun_phrases = [chunk.text.lower() for chunk in doc.noun_chunks]
most_common_nps = Counter(noun_phrases).most_common(10)

print("\n------ Most Common Noun Phrases ------")
for phrase, freq in most_common_nps:
    print(f"{phrase} ({freq})")

# 5. Parse Tree Visualization
# Function to convert spacy parse to nltk Tree
def create_tree(token):
    if token.n_lefts + token.n_rights > 0:
        return Tree(token.orth_, [create_tree(child) for child in
token.children])
    else:
        return token.orth_

print("\n------ Parse Trees for 5 Sentences ------")
for i, sent in enumerate(sentences[:5]):
    print(f"\nSentence {i+1}: {sent.text}")
    root = [token for token in sent if token.head == token][0]
    tree = create_tree(root)
    tree.pretty_print()
```

Output

```
------ Sentence Tokenization ------
1. Summary of Operation SINDOOR
On 22 April, Pakistan based terrorist group "The Resistance Front" (TRF)
perpetrated a devastating attack in Pahalgam, India.
2. In this attack 26 innocent tourists were killed in cold blood after
segregating them based on their religion.
3. TRF is an offshoot of the well known Pakistan backed terrorist
organisation Lashkar-e-Taiba.
4. Post the attack, TRF claimed the responsibility for this attack not
once but twice within a few hours.
5. Pakistan's subsequent refusal to acknowledge or curb these terrorist
networks compelled India to take a responsible but resolute action.
6. In response, on the night of 7-8 May 2025, the Government of India
executed "Operation Sindoor".
7. The response was non escalatory, precise and targeted terrorist
training camps at nine different locations within Pakistan and Pakistan
Occupied Kashmir.
```

8. No military targets were engaged.
9. However, in the early hours of 8 May, Pakistan, in an escalated response launched coordinated drone and missile strikes targeting over a dozen Indian military installations across the Northern and Western theatres, including Srinagar, Jammu, Pathankot, Amritsar, Ludhiana, Bathinda and Bhuj.
10. India’s robust Integrated Counter-drone Grid and layered Air Defence systems intercepted these attacks, recovering debris conclusively traced to Pakistani origin.

11. Following these provocations, India conducted precision strikes against Pakistani Air Defence systems at a number of locations in Pakistan.
12. These strikes were deliberately confined to the neutralization of systems that had facilitated the earlier Pakistani assault and were executed under the guiding principle of “equal intensity in the same domain.”
13. By targeting only those installations directly involved in the aggression, India balanced the imperative of deterrence with its overarching commitment to de-escalation.
14. Concurrently, along the Line of Control in Jammu & Kashmir, Pakistan escalated to unprovoked mortar and heavy-calibre artillery fire into civilian areas in which sixteen innocent lives were lost, including three women and five children.
15. Here too, India was compelled to respond in equal proportion with mortar and artillery fire.
16. Indian Armed forces reiterate their commitment to non-esclation, however, any attempts by Pakistan to escalate will be responded firmly.

------ POS Tagging ------

Summary	PROPN
of	ADP
Operation	PROPN
SINDOOR	PROPN
	SPACE
On	ADP
22	NUM
April	PROPN
,	PUNCT
Pakistan	PROPN
based	VERB
terrorist	ADJ
group	NOUN
“	PUNCT
The	DET
Resistance	PROPN
Front	PROPN
”	PUNCT
(	PUNCT
TRF	PROPN
)	PUNCT
perpetrated	VERB

a DET
devastating ADJ
attack NOUN
in ADP
Pahalgam PROPN
, PUNCT
India PROPN
. PUNCT
In ADP
this DET
attack NOUN
26 NUM
innocent ADJ
tourists NOUN
were AUX
killed VERB
in ADP
cold ADJ
blood NOUN
after ADP
segregating VERB
them PRON
based VERB
on ADP
their PRON
religion NOUN
. PUNCT
TRF PROPN
is AUX
an DET
offshoot NOUN
of ADP
the DET
well ADV
known VERB
Pakistan PROPN
backed VERB
terrorist ADJ
organisation NOUN
Lashkar PROPN
- PUNCT
e PROPN
- PROPN
Taiba PROPN
. PUNCT
Post VERB
the DET
attack NOUN
, PUNCT
TRF PROPN
claimed VERB
the DET
responsibility NOUN
for ADP

this	DET
attack	NOUN
not	PART
once	ADV
but	CCONJ
twice	ADV
within	ADP
a	DET
few	ADJ
hours	NOUN
.	PUNCT
Pakistan	PROPN
’s	PART
subsequent	ADJ
refusal	NOUN
to	PART
acknowledge	VERB
or	CCONJ
curb	VERB
these	DET
terrorist	ADJ
networks	NOUN
compelled	VERB
India	PROPN
to	PART
take	VERB
a	DET
responsible	ADJ
but	CCONJ
resolute	ADJ
action	NOUN
.	PUNCT
In	ADP
response	NOUN
,	PUNCT
on	ADP
the	DET
night	NOUN
of	ADP
7-8	NOUN
May	PROPN
2025	NUM
,	PUNCT
the	DET
Government	PROPN
of	ADP
India	PROPN
executed	VERB
“	PUNCT
Operation	PROPN
Sindoor	PROPN
”	PUNCT
.	PUNCT
The	DET

```
response    NOUN
was   AUX
non   ADJ
escalatory  ADJ
,     PUNCT
precise     ADJ
and   CCONJ
targeted    VERB
terrorist   ADJ
training    NOUN
camps NOUN
at    ADP
nine  NUM
different   ADJ
locations   NOUN
within      ADP
Pakistan    PROPN
and   CCONJ
Pakistan    PROPN
Occupied    PROPN
Kashmir     PROPN
.     PUNCT
No    DET
military    ADJ
targets     NOUN
were  AUX
engaged     VERB
.     PUNCT
However     ADV
,     PUNCT
in    ADP
the   DET
early ADJ
hours NOUN
of    ADP
8     NUM
May   PROPN
,     PUNCT
Pakistan    PROPN
,     PUNCT
in    ADP
an    DET
escalated   VERB
response    NOUN
launched    VERB
coordinated ADJ
drone NOUN
and   CCONJ
missile     NOUN
strikes     NOUN
targeting   VERB
over  ADP
a     DET
dozen NOUN
```

Indian ADJ
military ADJ
installations NOUN
across ADP
the DET
Northern ADJ
and CCONJ
Western ADJ
theatres NOUN
, PUNCT
including VERB
Srinagar PROPN
, PUNCT
Jammu PROPN
, PUNCT
Pathankot PROPN
, PUNCT
Amritsar PROPN
, PUNCT
Ludhiana PROPN
, PUNCT
Bathinda PROPN
and CCONJ
Bhuj PROPN
. PUNCT
India PROPN
’s PART
robust ADJ
Integrated PROPN
Counter PROPN
- NOUN
drone NOUN
Grid PROPN
and CCONJ
layered ADJ
Air PROPN
Defence PROPN
systems NOUN
intercepted VERB
these DET
attacks NOUN
, PUNCT
recovering VERB
debris NOUN
conclusively ADV
traced VERB
to ADP
Pakistani ADJ
origin NOUN
. PUNCT

SPACE
Following VERB
these DET

provocations NOUN
, PUNCT
India PROPN
conducted VERB
precision NOUN
strikes NOUN
against ADP
Pakistani PROPN
Air PROPN
Defence PROPN
systems NOUN
at ADP
a DET
number NOUN
of ADP
locations NOUN
in ADP
Pakistan PROPN
. PUNCT
These DET
strikes NOUN
were AUX
deliberately ADV
confined VERB
to ADP
the DET
neutralization NOUN
of ADP
systems NOUN
that PRON
had AUX
facilitated VERB
the DET
earlier ADJ
Pakistani ADJ
assault NOUN
and CCONJ
were AUX
executed VERB
under ADP
the DET
guiding VERB
principle NOUN
of ADP
“ PUNCT
equal ADJ
intensity NOUN
in ADP
the DET
same ADJ
domain NOUN
. PUNCT
” PUNCT
By ADP

```
targeting   VERB
only  ADV
those DET
installations    NOUN
directly   ADV
involved   VERB
in    ADP
the   DET
aggression  NOUN
,     PUNCT
India PROPN
balanced   VERB
the   DET
imperative  NOUN
of    ADP
deterrence  NOUN
with  ADP
its   PRON
overarching NOUN
commitment  NOUN
to    ADP
de    NOUN
-     NOUN
escalation  NOUN
.     PUNCT
Concurrently     ADV
,     PUNCT
along ADP
the   DET
Line  PROPN
of    ADP
Control     PROPN
in    ADP
Jammu PROPN
&     CCONJ
Kashmir     PROPN
,     PUNCT
Pakistan    PROPN
escalated   VERB
to    ADP
unprovoked  ADJ
mortar      NOUN
and   CCONJ
heavy ADJ
-     PUNCT
calibre     NOUN
artillery   NOUN
fire  NOUN
into  ADP
civilian    ADJ
areas NOUN
in    ADP
which PRON
sixteen     NUM
```

```
innocent    ADJ
lives NOUN
were  AUX
lost  VERB
,     PUNCT
including   VERB
three NUM
women NOUN
and   CCONJ
five  NUM
children    NOUN
.     PUNCT
Here  ADV
too   ADV
,     PUNCT
India PROPN
was   AUX
compelled   VERB
to    PART
respond     VERB
in    ADP
equal ADJ
proportion  NOUN
with  ADP
mortar      NOUN
and   CCONJ
artillery   NOUN
fire  NOUN
.     PUNCT
Indian      ADJ
Armed PROPN
forces      NOUN
reiterate   VERB
their PRON
commitment  NOUN
to    ADP
non   ADJ
-     NOUN
esclation   NOUN
,     PUNCT
however     ADV
,     PUNCT
any   DET
attempts    NOUN
by    ADP
Pakistan    PROPN
to    PART
escalate    VERB
will  AUX
be    AUX
responded   VERB
firmly      ADV
.     PUNCT
```

```
      SPACE

------ Named Entities ------
22 April    DATE
Pakistan    GPE
The Resistance Front   WORK_OF_ART
TRF   ORG
Pahalgam    GPE
India GPE
26    CARDINAL
TRF   ORG
Pakistan    GPE
Lashkar-e-Taiba   PERSON
TRF   ORG
a few hours TIME
Pakistan    GPE
India GPE
the night   TIME
May 2025    DATE
the Government of India      ORG
Operation SindoorWORK_OF_ART
nine  CARDINAL
Pakistan    GPE
Pakistan    GPE
Kashmir     LOC
the early hours   TIME
8 May DATE
Pakistan    GPE
over a dozen      CARDINAL
Indian      NORP
Northern    NORP
Srinagar    GPE
Jammu GPE
Amritsar    PERSON
Ludhiana    GPE
Bathinda    ORG
Bhuj  GPE
India GPE
Integrated Counter-drone     PERSON
Air Defence ORG
Pakistani   NORP
India GPE
Pakistani Air Defence  ORG
Pakistan    GPE
Pakistani   NORP
India GPE
the Line of Control    FAC
Jammu & Kashmir   ORG
Pakistan    GPE
sixteen     CARDINAL
three CARDINAL
five  CARDINAL
India GPE
Indian Armed      ORG
```

Pakistan GPE

------ Most Common Noun Phrases ------
india (6)
pakistan (6)
trf (3)
this attack (2)
jammu (2)
- (2)
summary (1)
operation sindoor (1)
22 april (1)
pakistan based terrorist group (1)

------ Parse Trees for 5 Sentences ------

```
Sentence 1: Summary of Operation SINDOOR
On 22 April, Pakistan based terrorist group “The Resistance Front” (TRF) perpetrated a devastating attack in Pahalgam, India.
                                                                perpetrated
                                                                  |
                                                                  |
                                                                  |
```

Sentence 3: TRF is an offshoot of the well known Pakistan backed terrorist organisation Lashkar-e-Taiba.

```
                          is
  _________________________|______
 |   |                       offshoot
 |   |   ______________________|___________
 |   |   |         of                      backed
 |   |   |         |                         |
 |   |   |      Pakistan                   Lashkar
 |   |   |    _____|_______        ____________|_______________
 |   |   |   |           known    |      |     |      |   organisation
 |   |   |   |             |      |      |     |      |        |
TRF  .   an the           well    -      e     -    Taiba  terrorist
```

Sentence 4: Post the attack, TRF claimed the responsibility for this attack not once but twice within a few hours.

```
                    claimed
  ____________________|_____________________________________
 |   |   |    |                  |                      once
 |   |   |    |                  |                 _______|_____
 |   |   |    |            responsibility         |           twice
 |   |   |    |          __________|__________     |     _________|_____
 |   |   |   Post      |                    for    |   |             within
 |   |   |    |        |                     |     |   |                |
 |   |   |  attack     |                   attack  |   |             hours
 |   |   |    |        |                     |     |   |          _____|_____
 ,  TRF  .   the      the                    this  not but        a          few
```

Sentence 5: Pakistan’s subsequent refusal to acknowledge or curb these terrorist networks compelled India to take a responsible but resolute action.

```
                                                compelled
   ____________________________________________|____________________________________
  |    |                      |                                |                    take
  |    |                      |                                |              _______|____
  |    |                   refusal                             |             |          action
  |    |        ________________|________                     |             |    ________|________
  |    |      |      Pakistan         acknowledge             networks      |   |             responsible
  |    |      |        |          ________|_________       _____|_________   |   |         ________|_________
India  .  subsequent    ’s       to        or        curb   these          terrorist  to  a        but              resolute
```

Text Classification using Machine Learning (25 Marks)

Using the **Reuters Corpus**:

1. Convert text to numerical features using:
    - Bag of Words
    - TF-IDF
2. Train a classifier (Naïve Bayes / Logistic Regression).
3. Evaluate using:
    - Accuracy
    - Precision
    - Recall
    - F1-score
4. Compare performance of BoW vs TF-IDF.

Expected: Code, confusion matrix, and result interpretation.

Code

```
import nltk
import numpy as np
from nltk.corpus import reuters
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer,
TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

# Download required dataset
nltk.download('reuters')

# Load Reuters Dataset
documents = reuters.fileids()
texts = []
labels = []

# Using only single-label documents for simplicity
```

```python
for doc_id in documents:
    categories = reuters.categories(doc_id)
    if len(categories) == 1:
        texts.append(reuters.raw(doc_id))
        labels.append(categories[0])

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.2, random_state=42
)

# 1. Bag of Words (BoW)
bow_vectorizer = CountVectorizer(stop_words='english', max_features=5000)

X_train_bow = bow_vectorizer.fit_transform(X_train)
X_test_bow = bow_vectorizer.transform(X_test)

# Train Naive Bayes classifier
nb_bow = MultinomialNB()
nb_bow.fit(X_train_bow, y_train)

# Prediction
y_pred_bow = nb_bow.predict(X_test_bow)

# Evaluation
print("\n===== Bag of Words Results =====")

print("Accuracy:", accuracy_score(y_test, y_pred_bow))
print("Precision:", precision_score(y_test, y_pred_bow,
average='weighted'))
print("Recall:", recall_score(y_test, y_pred_bow, average='weighted'))
print("F1 Score:", f1_score(y_test, y_pred_bow, average='weighted'))

cm_bow = confusion_matrix(y_test, y_pred_bow)
print("\nConfusion Matrix (BoW):\n", cm_bow)

# 2. TF-IDF
tfidf_vectorizer = TfidfVectorizer(stop_words='english',
max_features=5000)
```

```python
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)

# Train Naive Bayes classifier
nb_tfidf = MultinomialNB()
nb_tfidf.fit(X_train_tfidf, y_train)

# Prediction
y_pred_tfidf = nb_tfidf.predict(X_test_tfidf)

# Evaluation
print("\n===== TF-IDF Results =====")

print("Accuracy:", accuracy_score(y_test, y_pred_tfidf))
print("Precision:", precision_score(y_test, y_pred_tfidf,
average='weighted'))
print("Recall:", recall_score(y_test, y_pred_tfidf, average='weighted'))
print("F1 Score:", f1_score(y_test, y_pred_tfidf, average='weighted'))

cm_tfidf = confusion_matrix(y_test, y_pred_tfidf)
print("\nConfusion Matrix (TF-IDF):\n", cm_tfidf)
```

Output

```
[nltk_data] Downloading package reuters to /root/nltk_data...
[nltk_data]   Package reuters is already up-to-date!
===== Bag of Words Results =====
Accuracy: 0.9066593886462883
Precision: 0.9123936233410512
Recall: 0.9066593886462883
F1 Score: 0.9027183441088731

Confusion Matrix (BoW):
 [[461   0   0 ...   0   0   0]
 [  0   4   0 ...   0   0   0]
 [  0   0   3 ...   0   0   0]
 ...
 [  0   0   0 ...   1   0   0]
 [  0   0   0 ...   0   1   0]
 [  0   0   0 ...   0   0   0]]
/usr/local/lib/python3.12/dist-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
===== TF-IDF Results =====
Accuracy: 0.8198689956331878
Precision: 0.7607431677293987
Recall: 0.8198689956331878
F1 Score: 0.7745586705979461

Confusion Matrix (TF-IDF):
 [[468   0   0 ...   0   0   0]
 [  5   0   0 ...   0   0   0]
 [  0   0   0 ...   0   0   0]
 ...
 [  2   0   0 ...   0   0   0]
 [  0   0   0 ...   0   0   0]
 [  0   0   0 ...   0   0   0]]
/usr/local/lib/python3.12/dist-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Sentiment Analysis (25 Marks)

1. Collect 200 product reviews (Amazon / Flipkart dataset).
2. Preprocess text.
3. Train a sentiment classifier (Positive/Negative).
4. Evaluate model performance.
5. Test with 5 custom input sentences.

Tools: **Scikit-learn**, **NLTK**
Expected: Source code, evaluation metrics, and discussion of errors.

```
import pandas as pd

file_path = r"/content/sample_data/amazon_reviews.csv"
df = pd.read_csv(file_path)
df.head()
```

	Unnamed: 0	asin	helpful	overall	reviewText	reviewTime	reviewerID	reviewerName
0	0	0528881469	[0, 0]	5.0	We got this GPS for my husband who is an (OTR)...	06 2, 2013	AO94DHGC771SJ	amazdnu
1	1	0528881469	[12, 15]	1.0	I'm a professional OTR truck driver, and I bou...	11 25, 2010	AMO214LNFCEI4	Amazon Customer
2	2	0528881469	[43, 45]	3.0	Well, what can I say. I've had this unit in m...	09 9, 2010	A3N7T0DY83Y4IG	C. A. Freeman
3	3	0528881469	[9, 10]	2.0	Not going to write a long review, even thought...	11 24, 2010	A1H8PY3QHMQQA0	Dave M. Shaw "mack dave"
4	4	0528881469	[0, 0]	1.0	I've had mine for a year and here's what we go...	09 29, 2011	A24EV6RXELQZ63	Wayne Smith

```
df = df[['reviewText', 'overall']]
df = df.dropna()

# Take first 200 reviews
df = df.head(200)
df.shape
```

```
(200, 2)
```

```python
df = df[df['overall'] != 3]

def label_sentiment(rating):
    if rating >= 4:
        return "Positive"
    else:
        return "Negative"

df['Sentiment'] = df['overall'].apply(label_sentiment)

df['Sentiment'].value_counts()
```

Sentiment	count
Positive	173
Negative	8

```python
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

nltk.download('stopwords')
nltk.download('punkt_tab')
nltk.download('wordnet')

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z]', ' ', text)
    tokens = nltk.word_tokenize(text)
    tokens = [lemmatizer.lemmatize(word) for word in tokens if word not in stop_words]
    return " ".join(tokens)

df['Clean_Text'] = df['reviewText'].apply(preprocess)

df[['reviewText','Clean_Text']].head()
```

	reviewText	Clean_Text
0	We got this GPS for my husband who is an (OTR)...	got gps husband otr road trucker impressed shi...
1	I'm a professional OTR truck driver, and I bou...	professional otr truck driver bought tnd truck...
3	Not going to write a long review, even thought...	going write long review even thought unit dese...
4	I've had mine for a year and here's what we go...	mine year got try route non truck route tellin...
5	I am using this with a Nook HD+. It works as d...	using nook hd work described hd picture samsun...

```
from sklearn.model_selection import train_test_split

X = df['Clean_Text']
y = df['Sentiment']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

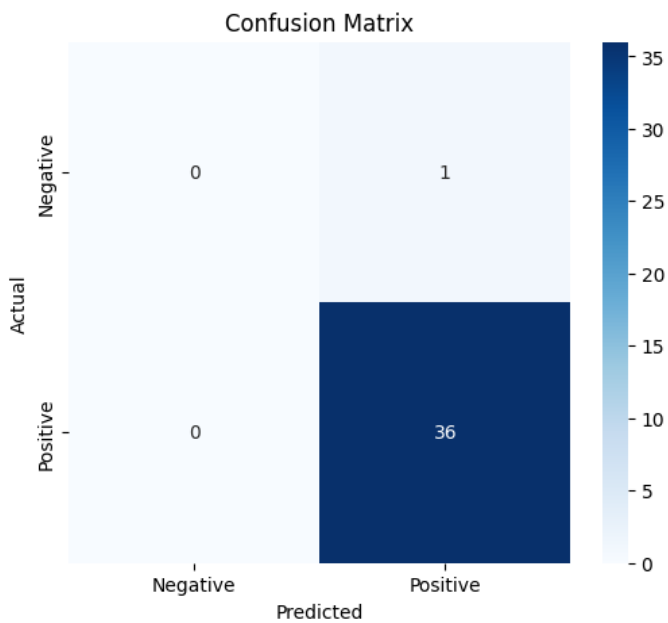
```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=3000)

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
```

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train_tfidf, y_train)

y_pred = model.predict(X_test_tfidf)
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, pos_label="Positive")
recall = recall_score(y_test, y_pred, pos_label="Positive")
f1 = f1_score(y_test, y_pred, pos_label="Positive")

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

```
Accuracy: 0.972972972972973
Precision: 0.972972972972973
Recall: 1.0
F1 Score: 0.9863013698630136
```

```python
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=["Negative","Positive"],
            yticklabels=["Negative","Positive"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



```python
custom_reviews = [
    "Completely wasteful and stopped working",
    "This product is absolutely grat and works perfectly",
    "Worst product I ever have",
    "It is okay but not worth the value",
    "Very happy with the purchase",
]

clean_custom = [preprocess(review) for review in custom_reviews]
vector_custom = vectorizer.transform(clean_custom)

predictions = model.predict(vector_custom)

for review, sentiment in zip(custom_reviews, predictions):
    print("Review:", review)
    print("Predicted Sentiment:", sentiment)
    print("-"*50)
```

```
Review: Completely wasteful and stopped working
Predicted Sentiment: Positive
--------------------------------------------------
Review: This product is absolutely grat and works perfectly
Predicted Sentiment: Positive
--------------------------------------------------
Review: Worst product I ever have
Predicted Sentiment: Positive
--------------------------------------------------
Review: It is okay but not worth the value
Predicted Sentiment: Positive
--------------------------------------------------
Review: Very happy with the purchase
Predicted Sentiment: Positive
--------------------------------------------------
```

N-gram Language Modeling (25 Marks)

1. Build unigram, bigram, and trigram models.
2. Calculate probabilities of sample sentences.
3. Implement simple text generation using the bigram model.
4. Compare model performance using perplexity.

Use any NLTK corpus.
Expected: Code and explanation of results.

```
!pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
```

```
import nltk
import random
import math
from collections import Counter, defaultdict
from nltk.util import ngrams
from nltk.corpus import gutenberg

nltk.download('gutenberg')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package gutenberg to /root/nltk_data...
[nltk_data]   Unzipping corpora/gutenberg.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
True
```

```
# Load text
text = gutenberg.raw('austen-emma.txt')

# Tokenize
tokens = nltk.word_tokenize(text.lower())
tokens = [word for word in tokens if word.isalpha()]

print("Total Tokens:", len(tokens))
```

```
Total Tokens: 157110
```

```python
unigram_counts = Counter(tokens)
total_unigrams = sum(unigram_counts.values())

def unigram_prob(word):
    return unigram_counts[word] / total_unigrams
```

```python
bigram_counts = Counter(ngrams(tokens, 2))
unigram_counts = Counter(tokens)

def bigram_prob(w1, w2):
    return bigram_counts[(w1, w2)] / unigram_counts[w1] if unigram_counts[w1] > 0 else 0
```

```python
trigram_counts = Counter(ngrams(tokens, 3))
bigram_counts = Counter(ngrams(tokens, 2))

def trigram_prob(w1, w2, w3):
    return trigram_counts[(w1, w2, w3)] / bigram_counts[(w1, w2)] if bigram_counts[(w1, w2)] > 0 else 0
```

```python
sentence = "emma was happy"
sentence_tokens = nltk.word_tokenize(sentence.lower())
sentence_tokens
```

```
['emma', 'was', 'happy']
```

```python
def sentence_prob_unigram(sentence_tokens):
    prob = 1
    for word in sentence_tokens:
        prob *= unigram_prob(word)
    return prob

print("Unigram Probability:", sentence_prob_unigram(sentence_tokens))
```

```
Unigram Probability: 6.535501076653638e-08
```

```python
def sentence_prob_bigram(sentence_tokens):
    prob = 1
    for w1, w2 in ngrams(sentence_tokens, 2):
        prob *= bigram_prob(w1, w2)
    return prob

print("Bigram Probability:", sentence_prob_bigram(sentence_tokens))
```

```
Bigram Probability: 6.406025546453392e-05
```

```python
def sentence_prob_trigram(sentence_tokens):
    prob = 1
    for w1, w2, w3 in ngrams(sentence_tokens, 3):
        prob *= trigram_prob(w1, w2, w3)
    return prob

print("Trigram Probability:", sentence_prob_trigram(sentence_tokens))
```

```
Trigram Probability: 0.0
```

```python
def generate_text_bigram(start_word, length=15):
    current_word = start_word
    output = [current_word]

    for _ in range(length):
        candidates = [(w2, bigram_counts[(current_word, w2)])
                      for (w1, w2) in bigram_counts if w1 == current_word]

        if not candidates:
            break

        words, weights = zip(*candidates)
        current_word = random.choices(words, weights=weights)[0]
        output.append(current_word)

    return " ".join(output)

print(generate_text_bigram("emma", 20))
```

```
emma could not fond of the turning to the injured her first reception of the merit what you that might have
```

```python
def perplexity_bigram(sentence_tokens):
    N = len(sentence_tokens)
    log_prob = 0

    for w1, w2 in ngrams(sentence_tokens, 2):
        prob = bigram_prob(w1, w2)
        if prob > 0:
            log_prob += math.log2(prob)

    return 2 ** (-log_prob / N)

print("Bigram Perplexity:", perplexity_bigram(sentence_tokens))
```

```
Bigram Perplexity: 24.992159157266112
```

```python
def perplexity_trigram(sentence_tokens):
    N = len(sentence_tokens)
    log_prob = 0

    for w1, w2, w3 in ngrams(sentence_tokens, 3):
        prob = trigram_prob(w1, w2, w3)
        if prob > 0:
            log_prob += math.log2(prob)

    return 2 ** (-log_prob / N)

print("Trigram Perplexity:", perplexity_trigram(sentence_tokens))
```

```
Trigram Perplexity: 1.0
```

Word Embeddings and Similarity (25 Marks)

1. Train Word2Vec model on any corpus OR use pre-trained embeddings.
2. Find:
    - Top 10 similar words for a given word
    - Odd word out from a list
3. Visualize embeddings using PCA or t-SNE.
4. Discuss semantic relationships observed.

Tools: Gensim / spaCy.
Expected: Code, visualization, and interpretation

```
!pip install nltk gensim scikit-learn matplotlib
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
```

```
import nltk
import gensim
import matplotlib.pyplot as plt
import numpy as np

from nltk.corpus import brown
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

nltk.download('brown')
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Unzipping corpora/brown.zip.
True
```

```
# Load Brown corpus sentences and Convert to lowercase
sentences = brown.sents()
sentences = [[word.lower() for word in sent] for sent in sentences]

print("Total Sentences:", len(sentences))
```

```
Total Sentences: 57340
```

```python
from gensim.models import Word2Vec

model = Word2Vec(
    sentences,
    vector_size=100,
    window=5,
    min_count=5,
    workers=4
)

print("Vocabulary size:", len(model.wv))

similar_words = model.wv.most_similar("class", topn=10)

print("10 similar words to 'class':")
for word, score in similar_words:
    print(word, "-", score)
```

```
10 similar words to 'class':
student - 0.9019176363945007
score - 0.8991893529891968
junior - 0.8983238339424133
crew - 0.8916490077972412
bones - 0.8894211053848267
selected - 0.8890603184700012
previous - 0.8887282013893127
stations - 0.8844298124313354
principal - 0.883928656578064
reports - 0.8825713992118835
```

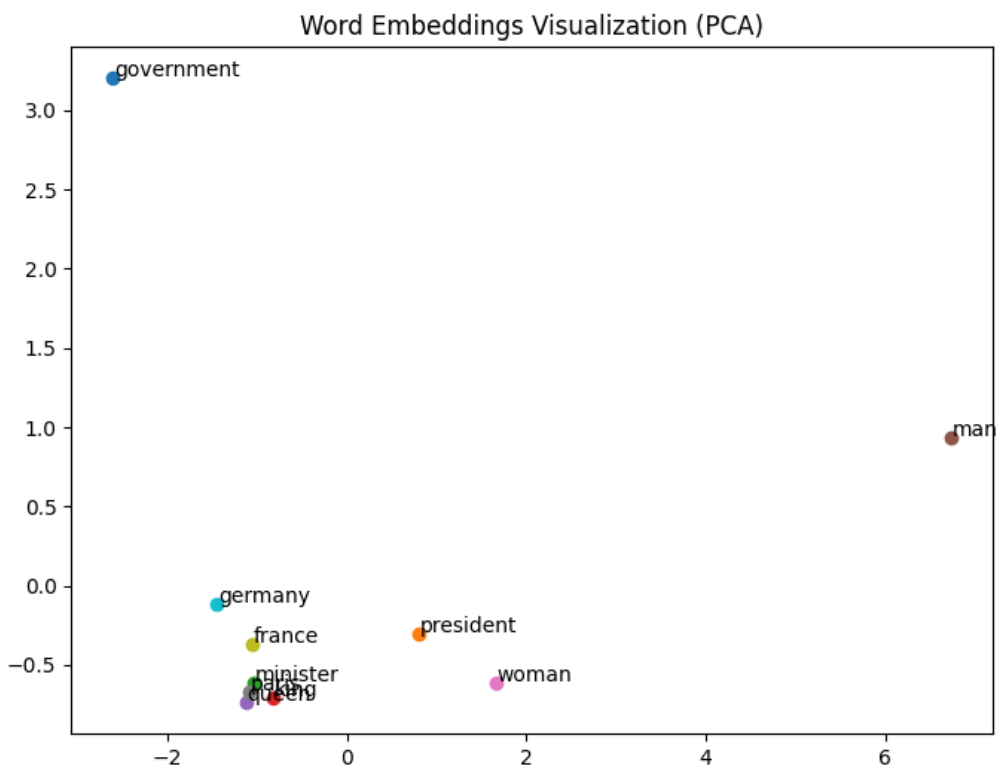
```python
words = ["france", "germany", "italy", "banana"]

odd_word = model.wv.doesnt_match(words)

print("Odd word out:", odd_word)
```

```
WARNING:gensim.models.keyedvectors:vectors for words {'banana'} are not present in the model, ignoring th
Odd word out: france
```

```python
model.wv.most_similar(positive=["king", "woman"], negative=["man"])
```

```
[('roman', 0.8934713006019592),
 ('won', 0.8928061127662659),
 ('opposed', 0.8925040364265442),
 ('chat', 0.8873541355133057),
 ('wedding', 0.8872602581977844),
 ('singing', 0.8871691226959229),
 ('breakfast', 0.886336088180542),
 ('danish', 0.8850337862968445),
 ('sister', 0.8844423890113831),
 ('sung', 0.884308934211731)]
```

```python
words_to_visualize = ["government", "president", "minister",
                      "king", "queen", "man", "woman",
                      "paris", "france", "germany"]

# Get vectors
vectors = np.array([model.wv[word] for word in words_to_visualize])

# Reduce dimensions
pca = PCA(n_components=2)
reduced_vectors = pca.fit_transform(vectors)

# Plot
plt.figure(figsize=(8,6))
for i, word in enumerate(words_to_visualize):
    x, y = reduced_vectors[i]
    plt.scatter(x, y)
    plt.text(x+0.01, y+0.01, word)

plt.title("Word Embeddings Visualization (PCA)")
plt.show()
```



Topic Modeling using LDA (25 Marks)

1. Use any large text dataset (e.g., news articles).
2. Preprocess text (lemmatization, stopword removal).
3. Apply Latent Dirichlet Allocation (LDA).
4. Extract top 10 words per topic.
5. Evaluate coherence score.
6. Interpret discovered topics.

Tools: Gensim / Scikit-learn.
Expected: Code, topics extracted, visualization, explanation.

```
!pip install nltk gensim scikit-learn pyLDAvis
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Collecting pyLDAvis
  Downloading pyLDAvis-3.4.1-py3-none-any.whl.metadata (4.2 kB)
```

```
import nltk
import re
import gensim
import numpy as np
import matplotlib.pyplot as plt

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.datasets import fetch_20newsgroups
from gensim import corpora
from gensim.models import CoherenceModel
from gensim.models.ldamodel import LdaModel

nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt')
```

```
# Load dataset
dataset = fetch_20newsgroups(remove=('headers', 'footers', 'quotes'))

documents = dataset.data
print("Total documents:", len(documents))
```

```
Total documents: 11314
```

```python
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    tokens = nltk.word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words and len(word) > 3]
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    return tokens

processed_docs = [preprocess(doc) for doc in documents]

print("Sample processed document:")
print(processed_docs[0][:20])
```

```
Sample processed document:
['wondering', 'anyone', 'could', 'enlighten', 'door', 'sport', 'looked', 'late', 'early', 'called',
```

```python
# Create dictionary
dictionary = corpora.Dictionary(processed_docs)

# Remove rare and very frequent words
dictionary.filter_extremes(no_below=15, no_above=0.5)

# Create Bag-of-Words corpus
corpus = [dictionary.doc2bow(doc) for doc in processed_docs]

print("Total unique tokens:", len(dictionary))
```

```
Total unique tokens: 5830
```

```python
num_topics = 5

lda_model = LdaModel(
    corpus=corpus,
    id2word=dictionary,
    num_topics=num_topics,
    random_state=42,
    passes=10,
    alpha='auto'
)
```

```python
for i in range(num_topics):
    print(f"\nTopic {i+1}:")
    words = lda_model.show_topic(i, topn=10)
    for word, prob in words:
        print(word, "->", round(prob, 4))
```

```
Topic 1:
drive -> 0.012
would -> 0.0092
card -> 0.0084
like -> 0.0079
problem -> 0.0075
system -> 0.0075
know -> 0.0067
work -> 0.0066
disk -> 0.0057
need -> 0.0057

Topic 2:
government -> 0.0083
state -> 0.007
armenian -> 0.0064
year -> 0.0056
space -> 0.0054
president -> 0.0051
national -> 0.0044
american -> 0.0043
public -> 0.0041
```

```python
coherence_model = CoherenceModel(
    model=lda_model,
    texts=processed_docs,
    dictionary=dictionary,
    coherence='c_v'
)

coherence_score = coherence_model.get_coherence()

print("Coherence Score:", coherence_score)
```

```
Coherence Score: 0.5304964333481232
```

Dependency Parsing and Chunking (25 Marks)

Using **spaCy**:

1. Perform dependency parsing on a paragraph.
2. Identify:
    - Subject-Verb relationships
    - Object relationships
3. Extract noun chunks.
4. Visualize dependency tree.
5. Explain syntactic structure of 5 sentences.

Expected: Parsed output + Analysis.

```
!pip install spacy
!python -m spacy download en_core_web_sm
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages
actions ement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from s
```

```
import spacy
from spacy import displacy

nlp = spacy.load("en_core_web_sm")


text = """
The government introduced new economic reforms to boost growth.
Several industry leaders praised the policy changes.
The prime minister announced a new development plan yesterday.
Consumers reported problems with rising inflation rates.
The central bank adjusted interest rates to stabilize the economy.
Many investors showed confidence in the financial reforms.
Researchers published a detailed report on economic performance.
"""
doc = nlp(text)
```

```python
for token in doc:
    print(token.text, "->", token.dep_, "->", token.head.text)
```

```
 -> dep -> .
The -> det -> minister
prime -> amod -> minister
minister -> nsubj -> announced
announced -> ROOT -> announced
a -> det -> plan
new -> amod -> plan
development -> compound -> plan
plan -> dobj -> announced
yesterday -> npadvmod -> announced
. -> punct -> announced

 -> dep -> .
Consumers -> nsubj -> reported
reported -> ROOT -> reported
problems -> dobj -> reported
with -> prep -> problems
rising -> amod -> rates
inflation -> compound -> rates
rates -> pobj -> with
. -> punct -> reported
```

```python
print("Subject-Verb Relationships:\n")

for token in doc:
    if token.dep_ in ("nsubj", "nsubjpass"):
        print("Subject:", token.text,
              "| Verb:", token.head.text)
```

```
Subject-Verb Relationships:

Subject: government | Verb: introduced
Subject: leaders | Verb: praised
Subject: minister | Verb: announced
Subject: Consumers | Verb: reported
Subject: investors | Verb: showed
Subject: Researchers | Verb: published
```

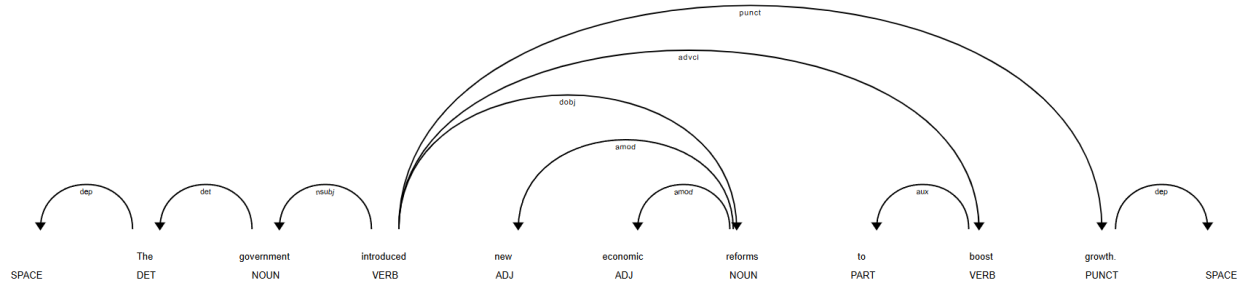
```python
print("\nObject Relationships:\n")

for token in doc:
    if token.dep_ in ("dobj", "pobj"):
        print("Object:", token.text,
              "| Head:", token.head.text)
```

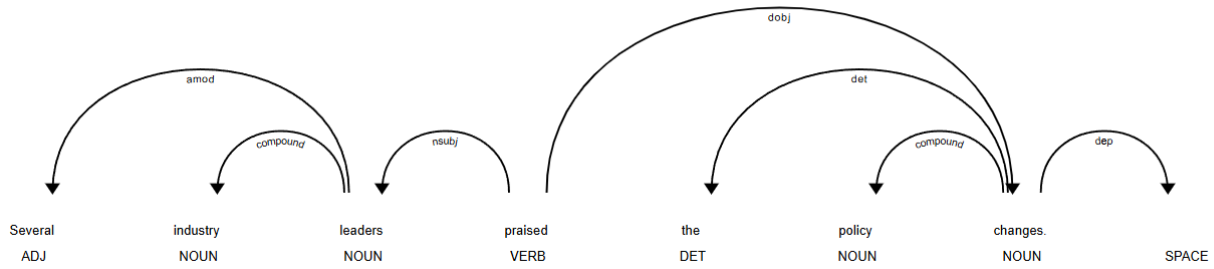
```
Object Relationships:

Object: reforms | Head: introduced
Object: growth | Head: boost
Object: changes | Head: praised
Object: plan | Head: announced
Object: problems | Head: reported
Object: rates | Head: with
Object: economy | Head: stabilize
Object: confidence | Head: showed
Object: reforms | Head: in
Object: report | Head: published
Object: performance | Head: on
```

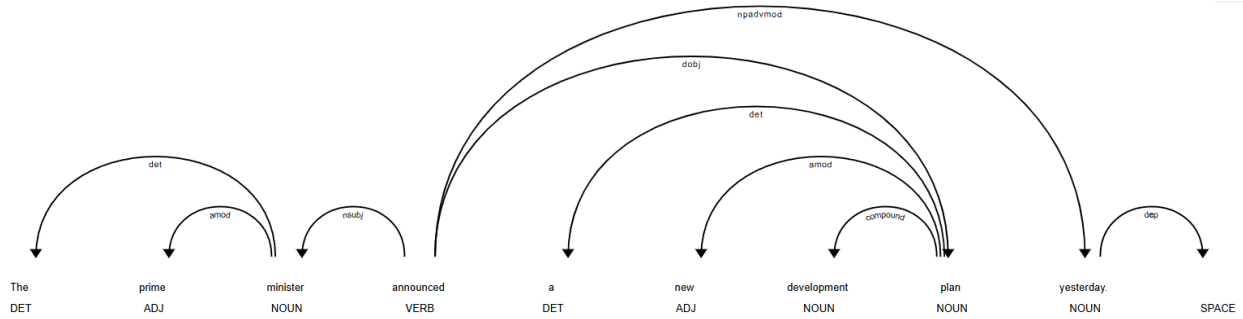
```python
print("\nNoun Chunks:\n")

for chunk in doc.noun_chunks:
    print(chunk.text)
```

```
Noun Chunks:


The government
new economic reforms
growth
Several industry leaders
the policy changes
The prime minister
a new development plan
Consumers
problems
rising inflation rates
The central bank adjusted interest rates
the economy
Many investors
confidence
the financial reforms
Researchers
a detailed report
economic performance
```

```python
displacy.render(doc, style="dep", jupyter=True)
```


punct
advcl
dobj
amod
dep
det
nsubj
amod
aux
dep
SPACE
The
DET
government
NOUN
introduced
VERB
new
ADJ
economic
ADJ
reforms
NOUN
to
PART
boost
VERB
growth.
PUNCT
SPACE


dobj
amod
det
compound
nsubj
compound
dep
Several
ADJ
industry
NOUN
leaders
NOUN
praised
VERB
the
DET
policy
NOUN
changes.
NOUN
SPACE


npadvmod
dobj
det
det
amod
amod
nsubj
compound
dep
The
DET
prime
ADJ
minister
NOUN
announced
VERB
a
DET
new
ADJ
development
NOUN
plan
NOUN
yesterday.
NOUN
SPACE

Spam Email Classification (25 Marks)

1. Use a publicly available spam dataset.
2. Preprocess text.
3. Extract features using TF-IDF.
4. Train:
   - Naïve Bayes
   - SVM
5. Compare models using Accuracy, Precision, Recall, F1-score.
6. Discuss overfitting/underfitting.

Expected: Code + Comparison table + Interpretation

```
import pandas as pd

file_path = r"/content/sample_data/enron_spam_data.csv"
df = pd.read_csv(file_path)
df.head()
```

	Message ID	Subject	Message	Spam/Ham	Date
0	0	christmas tree farm pictures	NaN	ham	1999-12-10
1	1	vastar resources , inc .	gary , production from the high island larger ...	ham	1999-12-13
2	2	calpine daily gas nomination	- calpine daily gas nomination 1 . doc	ham	1999-12-14
3	3	re : issue	fyi - see note below - already done .\nstella\...	ham	1999-12-14
4	4	meter 7268 nov allocation	fyi .\n-----...-----...	ham	1999-12-14

```
df['text'] = df['Subject'].fillna('') + " " + df['Message'].fillna('')
```

```
df['label'] = df['Spam/Ham'].map({'spam':1, 'ham':0})
df['label'].value_counts()
```

label	count
1	17171
0	16545

```python
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

import re

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    tokens = nltk.word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words and len(word) > 2]
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    return " ".join(tokens)

df['clean_text'] = df['text'].apply(preprocess)

df[['text','clean_text']].head()
```

	text	clean_text
0	christmas tree farm pictures	christmas tree farm picture
1	vastar resources , inc . gary , production fro...	vastar resource inc gary production high islan...
2	calpine daily gas nomination - calpine daily g...	calpine daily gas nomination calpine daily gas...
3	re : issue fyi - see note below - already done...	issue fyi see note already done stella forward...
4	meter 7268 nov allocation fyi .\n- - - - - - -...	meter nov allocation fyi forwarded lauri allen...

```python
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(max_features=5000)

X = vectorizer.fit_transform(df['clean_text'])
y = df['label']
```

```python
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```python
from sklearn.naive_bayes import MultinomialNB
nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)

nb_pred = nb_model.predict(X_test)
```

```python
from sklearn.svm import LinearSVC
svm_model = LinearSVC()
svm_model.fit(X_train, y_train)

svm_pred = svm_model.predict(X_test)
```

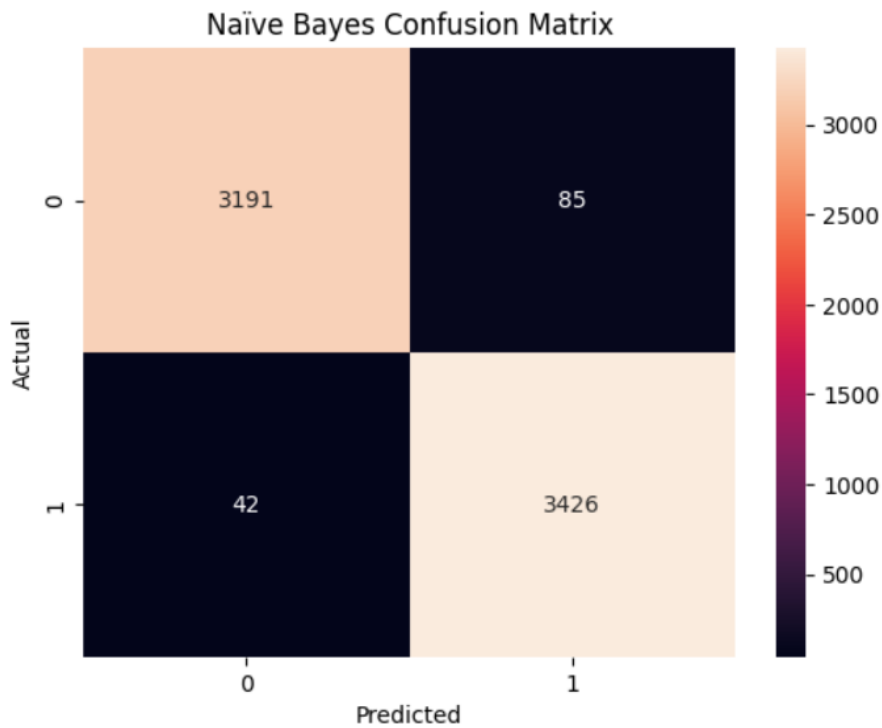
```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
print("Naïve Bayes Performance:")
print("Accuracy:", accuracy_score(y_test, nb_pred))
print("Precision:", precision_score(y_test, nb_pred))
print("Recall:", recall_score(y_test, nb_pred))
print("F1 Score:", f1_score(y_test, nb_pred))
```

```
Naïve Bayes Performance:
Accuracy: 0.9811684460260973
Precision: 0.9757903731130732
Recall: 0.9878892733564014
F1 Score: 0.9818025505086688
```

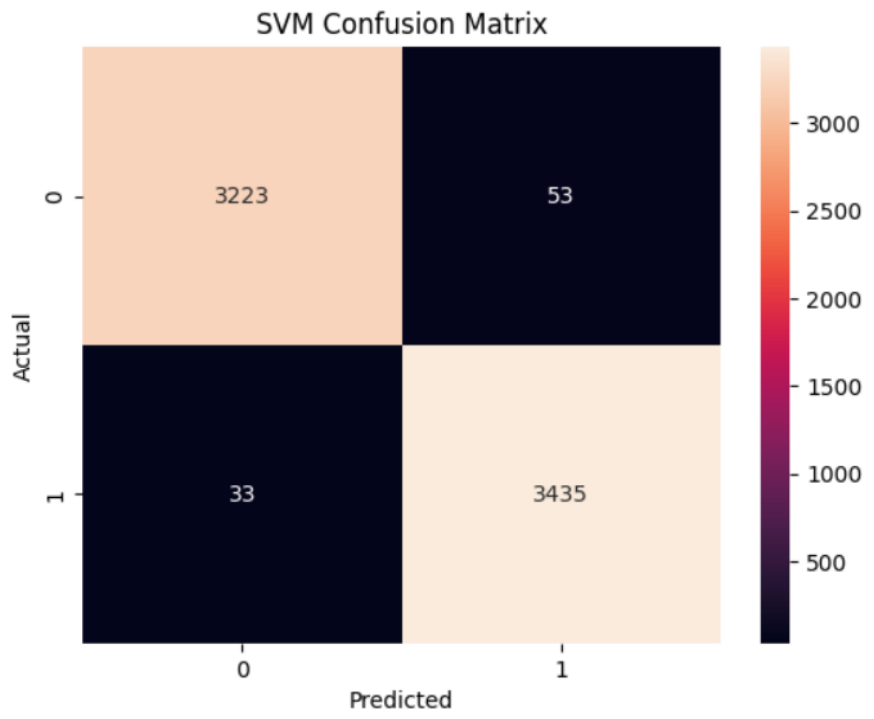
```python
print("SVM Performance:")
print("Accuracy:", accuracy_score(y_test, svm_pred))
print("Precision:", precision_score(y_test, svm_pred))
print("Recall:", recall_score(y_test, svm_pred))
print("F1 Score:", f1_score(y_test, svm_pred))
```

```
SVM Performance:
Accuracy: 0.9872479240806643
Precision: 0.9848050458715596
Recall: 0.990484429065744
F1 Score: 0.9876365727429557
```

```python
import matplotlib.pyplot as plt
cm_nb = confusion_matrix(y_test, nb_pred)

sns.heatmap(cm_nb, annot=True, fmt='d')
plt.title("Naïve Bayes Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```python
sns.heatmap(cm_svm, annot=True, fmt='d')
plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Named Entity Recognition on Custom Dataset (25 Marks)

1. Create your own small corpus (minimum 20 paragraphs).
2. Annotate entities manually (Person, Location, Organization).
3. Train a simple NER model.
4. Evaluate using precision and recall.
5. Discuss limitations.

Tools: spaCy custom training pipeline.
Expected: Dataset + Model + Evaluation

```
TRAIN_DATA = [
    ("Elon Musk founded SpaceX in California.",
     {"entities": [(0, 9, "PERSON"), (18, 24, "ORG"), (28, 38, "GPE")]}),

    ("Sundar Pichai is the CEO of Google.",
     {"entities": [(0, 13, "PERSON"), (28, 34, "ORG")]}),

    ("Microsoft was established in Washington.",
     {"entities": [(0, 9, "ORG"), (28, 38, "GPE")]}),

    ("Narendra Modi visited New York.",
     {"entities": [(0, 13, "PERSON"), (22, 30, "GPE")]}),

    ("Apple released the new iPhone in London.",
     {"entities": [(0, 5, "ORG"), (34, 40, "GPE")]}),

    ("Jeff Bezos started Amazon in Seattle.",
     {"entities": [(0, 10, "PERSON"), (19, 25, "ORG"), (29, 36, "GPE")]}),

    ("Tesla operates factories in Germany.",
     {"entities": [(0, 5, "ORG"), (29, 36, "GPE")]}),

    ("Barack Obama was born in Hawaii.",
     {"entities": [(0, 12, "PERSON"), (25, 31, "GPE")]}),

    ("Facebook headquarters are in California.",
     {"entities": [(0, 8, "ORG"), (28, 38, "GPE")]}),
```

```python
import spacy
from spacy.training.example import Example
import random

# Create blank English model
nlp = spacy.blank("en")

# Add NER pipeline
ner = nlp.add_pipe("ner")

# Add labels
for _, annotations in TRAIN_DATA:
    for ent in annotations["entities"]:
        ner.add_label(ent[2])

# Training
optimizer = nlp.begin_training()

for epoch in range(30):
    random.shuffle(TRAIN_DATA)
    losses = {}

    for text, annotations in TRAIN_DATA:
        doc = nlp.make_doc(text)
        example = Example.from_dict(doc, annotations)
        nlp.update([example], losses=losses)

    print(f"Epoch {epoch+1}, Losses: {losses}")
```

```
Epoch 1, Losses: {'ner': np.float32(72.370056)}
Epoch 2, Losses: {'ner': np.float32(38.925243)}
Epoch 3, Losses: {'ner': np.float32(21.685001)}
Epoch 4, Losses: {'ner': np.float32(18.432344)}
Epoch 5, Losses: {'ner': np.float32(5.2721095)}
Epoch 6, Losses: {'ner': np.float32(6.62794)}
Epoch 7, Losses: {'ner': np.float32(2.4115298)}
Epoch 8, Losses: {'ner': np.float32(2.6299508)}
Epoch 9, Losses: {'ner': np.float32(2.1012726)}
Epoch 10, Losses: {'ner': np.float32(0.04217106)}
Epoch 11, Losses: {'ner': np.float32(0.00011429691)}
Epoch 12, Losses: {'ner': np.float32(1.2004945e-05)}
Epoch 13, Losses: {'ner': np.float32(5.803951e-06)}
Epoch 14, Losses: {'ner': np.float32(2.2594616e-06)}
Epoch 15, Losses: {'ner': np.float32(1.2434579e-06)}
Epoch 16, Losses: {'ner': np.float32(1.1935638e-06)}
Epoch 17, Losses: {'ner': np.float32(4.4904004e-07)}
Epoch 18, Losses: {'ner': np.float32(2.4786315e-05)}
Epoch 19, Losses: {'ner': np.float32(3.815914e-07)}
Epoch 20, Losses: {'ner': np.float32(2.3964571e-06)}
```

```python
test_text = "Elon Musk visited Microsoft in Germany."

doc = nlp(test_text)

for ent in doc.ents:
    print(ent.text, ent.label_)
```

```
Elon Musk PERSON
Microsoft ORG
Germany GPE
```

```python
from spacy.scorer import Scorer

scorer = Scorer()

examples = []
for text, annotations in TRAIN_DATA:
    doc = nlp.make_doc(text)
    example = Example.from_dict(doc, annotations)
    pred_doc = nlp(text)
    example = Example(pred_doc, example.reference)
    examples.append(example)

scores = scorer.score(examples)

print("Precision:", scores["ents_p"])
print("Recall:", scores["ents_r"])
print("F1 Score:", scores["ents_f"])
```

```
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
```

Text Similarity and Plagiarism Detection (25 Marks)

1. Take 5 documents (minimum 300 words each).
2. Compute:
   - Cosine similarity (TF-IDF)
   - Jaccard similarity
3. Identify most similar document pairs.
4. Create a similarity matrix.
5. Discuss potential plagiarism cases.

Expected: Code + Matrix + Analysis.

```
!pip install python-docx scikit-learn nltk pandas seaborn matplotlib
```

```
Collecting python-docx
  Downloading python_docx-1.2.0-py3-none-any.whl (252 kB)
     ---------------------------------------- 253.0/253.0 kB 3.1 MB/s eta 0:00:00
Requirement already satisfied: scikit-learn in c:\users\admin\appdata\local\programs\python\pyth
Requirement already satisfied: nltk in c:\users\admin\appdata\local\programs\python\python311\li
Requirement already satisfied: pandas in c:\users\admin\appdata\local\programs\python\python311\
Requirement already satisfied: seaborn in c:\users\admin\appdata\local\programs\python\python311
Requirement already satisfied: matplotlib in c:\users\admin\appdata\local\programs\python\python
Collecting lxml>=3.1.0
  Downloading lxml-6.0.2-cp311-cp311-win_amd64.whl (4.0 MB)
```

```
import os
import re
import nltk
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from docx import Document
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\ADMIN\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
True
```

```python
folder_path = r"C:\Users\Admin\Documents"

file_names = [
    "Impact of Extended Reality.docx",
    "Linear regression model_6717.docx",
    "Project Proposal.docx",
    "SoftwareRequirements.docx",
    "twitter api.docx"
]

def read_docx(file_path):
    doc = Document(file_path)
    return "\n".join([para.text for para in doc.paragraphs])

documents = []
doc_labels = []

for file in file_names:
    file_path = os.path.join(folder_path, file)
    text = read_docx(file_path)
    documents.append(text)
    doc_labels.append(file.replace(".docx", ""))

print("Loaded Documents:", len(documents))
```

```
Loaded Documents: 5
```

```python
def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    return text

clean_docs = [preprocess(doc) for doc in documents]
```

```
vectorizer = TfidfVectorizer(stop_words='english')
tfidf_matrix = vectorizer.fit_transform(clean_docs)
cosine_sim_matrix = cosine_similarity(tfidf_matrix)
cosine_df = pd.DataFrame(cosine_sim_matrix,
                         index=doc_labels,
                         columns=doc_labels)
cosine_df
```

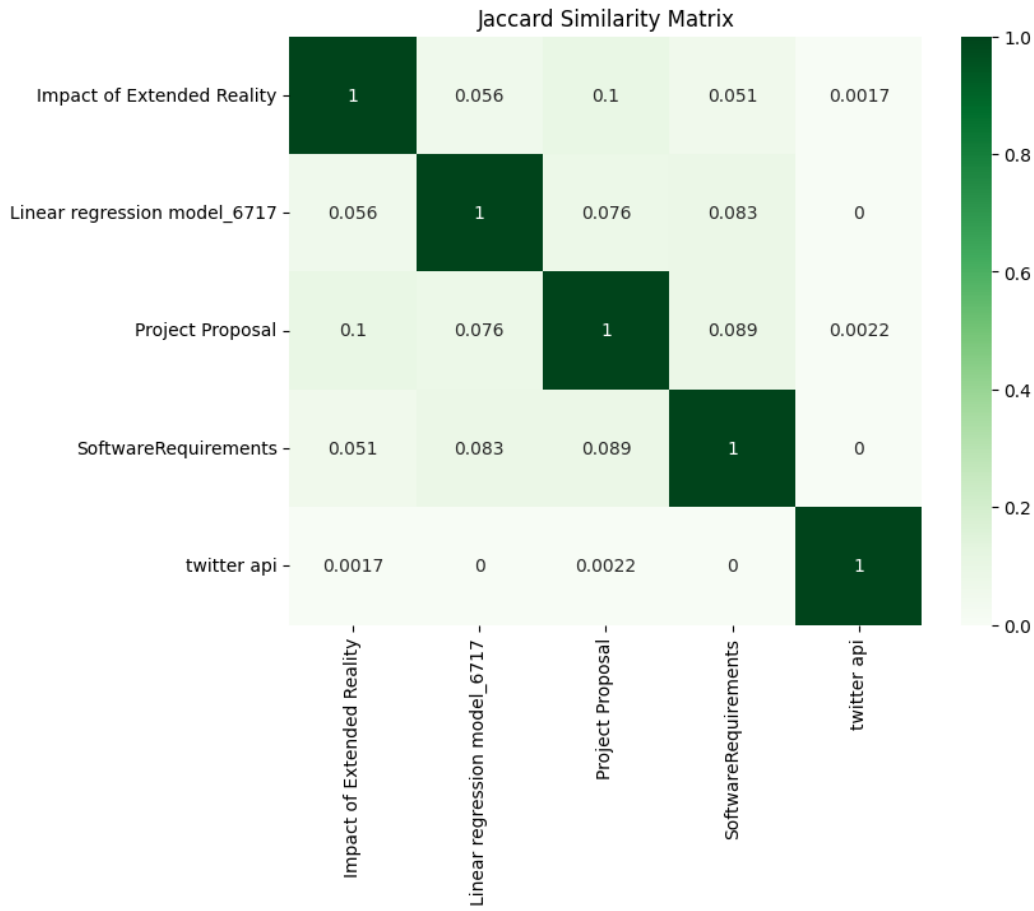
	Impact of Extended Reality	Linear regression model_6717	Project Proposal
Impact of Extended Reality	1.000000	0.039107	0.048832
Linear regression model_6717	0.039107	1.000000	0.065498
Project Proposal	0.048832	0.065498	1.000000
SoftwareRequirements	0.039888	0.019386	0.047740
twitter api	0.007898	0.000000	0.041028

```
def jaccard_similarity(doc1, doc2):
    set1 = set(nltk.word_tokenize(doc1))
    set2 = set(nltk.word_tokenize(doc2))
    return len(set1.intersection(set2)) / len(set1.union(set2))

jaccard_matrix = np.zeros((len(clean_docs), len(clean_docs)))

for i in range(len(clean_docs)):
    for j in range(len(clean_docs)):
        jaccard_matrix[i][j] = jaccard_similarity(clean_docs[i], clean_docs[j])

jaccard_df = pd.DataFrame(jaccard_matrix,
                          index=doc_labels,
                          columns=doc_labels)

jaccard_df
```

	Impact of Extended Reality	Linear regression model_6717	Project Proposal	SoftwareRequirements
Impact of Extended Reality	1.000000	0.056135	0.101319	0.051262
Linear regression model_6717	0.056135	1.000000	0.076106	0.082569
Project Proposal	0.101319	0.076106	1.000000	0.089005
SoftwareRequirements	0.051262	0.082569	0.089005	1.000000
twitter api	0.001732	0.000000	0.002237	0.000000

```
plt.figure(figsize=(8,6))
sns.heatmap(cosine_df, annot=True, cmap="Blues")
plt.title("Cosine Similarity Matrix (TF-IDF)")
plt.show()
```

```
plt.figure(figsize=(8,6))
sns.heatmap(jaccard_df, annot=True, cmap="Greens")
plt.title("Jaccard Similarity Matrix")
plt.show()
```




```
np.fill_diagonal(cosine_sim_matrix, 0)

max_index = np.unravel_index(np.argmax(cosine_sim_matrix), cosine_sim_matrix.shape)

print("Most Similar Pair (Cosine):")
print(doc_labels[max_index[0]], "<--->", doc_labels[max_index[1]])
print("Similarity Score:", cosine_sim_matrix[max_index])
```

```
Most Similar Pair (Cosine):
Linear regression model_6717 <---> Project Proposal
Similarity Score: 0.06549827762503059
```